

Script-Controlled Access Control List (ACL) Execution Journey Map

Phase 1: Ideation & Requirements

	1. Entice / Initiate Trigger Access Request	2. Enter / Intercept Rule Lookup & Matching	3. Engage / Evaluate Execute Server JS Logic	4. Exit / Enforce Grant or Lock Access	5. Extend / Audit Security Debug & Logging
Steps What occurs at each phase?	User opens form, submits record, or triggers REST API endpoint.	Platform intercepts request & queries <code>sys_security_acl</code> table.	Server evaluates script: <code>answer = evaluateAccess()</code> .	Security Engine renders fields as read/write or locks access.	Evaluation decision is saved to instance security logs.
Interactions People, Places, Things	Fulfiller / REST Integration Client in Workspace/API	ServiceNow Security Engine & ACL Context Evaluator.	Server JavaScript Engine, <code>GlideRecord</code> , & <code>gs.API</code> .	UI Form controls & REST API status payload (200 / 403).	System Security Log & Security Debugger tool.
Goals & Motivations Primary intent & expectations	Complete business tasks quickly without unneeded access blocks.	Enforce zero-trust security before data reaches the client.	Evaluate dynamic context checks with minimal query latency.	Prevent post-closure record modifications and data breaches.	Ensure 100% data governance and full audit trail compliance.
Positive Moments Successes & smooth flows	Instant, context-aware field visibility for authorized staff.	Fast ACL evaluation on Table and Field hierarchy levels.	Dynamic assignment rules and roles checked accurately.	Tamper-proof protection across UI Workspace and REST APIs.	Clear execution trace in <code>Debug 'Security Rules'</code> tool.
Negative Moments Pains & obstacles	User confusion when fields become read-only unexpectedly.	Conflicting wildcard (*) ACL rules executing together.	Unoptimized script queries slowing list view loading times.	Accidental lockouts of legitimate fulfillers due to bad logic.	Complex troubleshooting across parent-child table hierarchies.
Areas of Opportunity Improvements & solutions	Add clear UI info-hints explaining field restriction reasons.	Consolidate redundant ACL rules into central Script Includes.	Optimize scripts using <code>gs.hasField()</code> instead of <code>GlideRecord</code> .	Provide custom error messages for unauthorized API calls.	Automate ACL regression tests using Automated Test Framework.